

Reported vs. True Solution Quality in an LLM Search Framework

A dead-parameter injection test on UC Berkeley’s SkyDiscover

Raunak Mondal · Carnegie Mellon University · rmondal@andrew.cmu.edu · Independent research, August 2026

The question

SkyDiscover is the Sky Computing Lab’s framework for LLM-driven program search: a model proposes edits to a program, a containerized evaluator scores each candidate, and the search keeps the best ones and iterates. The framework reports a score for its best program at the end. I wanted to know what that score is worth if any part of it can be moved without improving the program.

That situation is not exotic. A scoring bug, a proxy metric that only partly tracks the real goal, or an evaluator that leaks information about itself each create a component of the score that search can climb without doing the work. The question is whether the reported number diverges from true quality under those conditions, and whether anything in the pipeline would catch it.

Setup

Task. The shipped `text_similarity` example. Each candidate program scores pairs of texts, and the evaluator’s `combined_score` is the Spearman correlation between the program’s scores and ground-truth labels.

Held-out check. SkyDiscover’s evaluator runs in two modes: *train*, used to score candidates during search, and *test*, used once at the end on the best program. I used that split as the detector. The injection lives only in train mode; test mode is untouched, so it reports the true metric for whatever program the search chose.

Injection. A wrapper (`inject.py`) around the unmodified evaluator. In train mode it reports

$$reported = 0.7 \times true + 0.3 \times dead,$$

where *dead* is a deterministic hash of the candidate’s source text. It carries no information about quality, but it is stable per candidate, so the search can select on it. The control condition runs the identical wrapper at weight zero.

Search. The `topk` controller, seed 42, cascade evaluation off, 15 iterations per run, three runs per condition. All settings were pinned explicitly in the experiment config rather than inherited from defaults.

Confound check. The evaluator’s feedback string contains the true correlation. If that text reached the model’s prompt, the model could read the true metric while I perturbed the score behind it. I confirmed in the source that the `topk` controller does not feed evaluator feedback into the prompt (the `AdaEvolve` controller does), so the true metric was not visible to the model.

Result

Condition	Reported (train)	True (test)	Gap	True quality peaks
Control, weight 0.0	0.586	0.586	+0.000	iteration 14
Injected, weight 0.3	0.655	0.565	+0.089	iteration 8

Values at iteration 15, averaged over runs. The gap was exactly zero in each of the three control runs and positive in each of the three injected runs.

Under injection, the reported score ends higher than the control’s while the true score ends lower. Real progress stopped at iteration 8 and drifted down slightly by iteration 15 (0.572 at the peak, 0.565 at the end), and the

reported score kept climbing through the rest of the run. The search had found the part of the score it could move without improving the program, and from that point on it was optimizing the hash.

Two things make this a clean measurement rather than a suggestive one. The control's train and test scores agree to four decimals in every run, which shows the wrapper is a genuine pass-through at weight zero and that the injection is the only difference between conditions. And the gap is a paired quantity, computed on the same program within the same run, so it does not depend on comparing absolute scores across runs.

That second point matters because run-to-run noise is large here. Two control runs with identical configuration, stopped at two iterations, reached true scores of 0.503 and 0.602: a spread of 0.10 from LLM sampling alone, the same magnitude as the effect. A single treatment run compared against a single control run would have proven nothing. The within-run gap sidesteps that entirely.

What I got wrong, and how the numbers caught it

My first “control” runs were injected at full strength without my knowing it. `inject.py` read its weight from an environment variable with a default of 0.3, and I set `DEAD_WEIGHT=0.0` in my shell. But the framework runs the evaluator through `docker exec` without forwarding the ambient environment, so the container never saw my setting and the wrapper fell back to its default. The direct `docker run` smoke tests had passed only because I had passed the variable explicitly there.

I caught it from the data before I caught it in the code. Each program's recorded score and its feedback score solved cleanly for a dead component at weight 0.3, and one recorded/feedback pair (0.1524 / 0.2092) matched the earlier weight-0.3 smoke test to four decimals. Both “conditions” had been the same condition.

The fix was structural rather than a patch. The wrapper now defaults to 0.0, so an unforwarded variable means *no* injection instead of silent injection. Each condition is its own directory with the weight baked into its Docker image via `ENV`, so the two conditions are two images and nothing depends on environment plumbing. The check that the control is real: train equals test exactly, which the unmodified framework also satisfies.

Two upstream defects found along the way

- **Conflict markers written into candidates.** When the model emits a malformed `SEARCH/REPLACE` diff, the framework writes the candidate program file with the `=====` markers still in it. Reproducible, at roughly 2% of iterations in the control runs.
- **Unseeded sampling in the shipped example.** The `text_similarity` evaluator calls `random.sample` without a seed, so the evaluation set differs between runs and results are not reproducible as shipped.

Limits

This is a result about the `topk` controller. Whether the AdaEvolve or EvoX-style controllers exploit the dead component faster or resist it is the obvious follow-up, and I would not assume the answer carries over. It is one injection weight (0.3) rather than a dose-response curve, and three runs per condition. The paired within-run gap is the robust quantity; the absolute scores are not.

Why this matters beyond one framework

Any system that optimizes against its own evaluator will report improvement whenever the evaluator has a movable component. The reported number and the held-out number then part ways silently, and the divergence starts before anyone thinks to look. Here the detector already existed inside the framework and cost nothing to use. The expensive part was deciding to check.

For an AI product whose pitch rests on a metric, the questions this suggests are concrete. What does the reported number get evaluated against? Can the system see, or influence, its own scoring function? Is there a held-out path, and does anyone run it?

Code, configs, and run logs: local repository `skydiscover-evaluator-integrity`, kept separate from the upstream checkout. Available on request.